

UNIVERSIDAD NACIONAL DEL SANTA

FACULTAD DE INGENIERIA

Escuela Académica Profesional De Ingeniería De Sistemas E
Informática



"SEMANA 08 – REPRESENTACIONES CROMOSÓMICAS"

ALUMNO:

- Estefanero Palacios, Jael Andres

DOCENTE:

- Lopez Heredia, Johan Max Alexander

CURSO:

- Algoritmos Evolutivos

CICLO: IX

SEMESTRE: 2025-I

NUEVO CHIMBOTE – PERÚ

2025

ÍNDICE

1	Comparación de las representaciones	1
1.1	Representación Binaria	1
1.2	Representación Real	1
1.3	Representación Permutacional.....	2
2	Actividades Realizadas	3
2.1	Actividad 1: Análisis Comparativo.....	3
2.1.1	Resultados	3
2.1.2	Conclusiones	3
2.2	Actividad 2: Modificación de Fitness	3
2.2.1	Resultados	4
2.2.2	Conclusiones	4
2.3	Actividad 3: Nuevo Operador Genético	4
2.3.1	Resultados	5
2.3.2	Conclusiones	5
2.4	Actividad 4: Restricciones Adicionales	5
2.4.1	Resultados	6
2.4.2	Conclusiones	6
2.5	Actividad 5: Visualización.....	6
2.5.1	Resultados	6
2.6	Actividad 6: Problema Extendido.....	7
2.6.1	¿Qué cambios necesitas hacer en el cromosoma?.....	7
2.6.2	¿Cómo afecta esto a la convergencia del algoritmo?.....	8
2.6.3	Resultados	8

2.6.4 Conclusiones	8
3 Preguntas de reflexión.....	9
3.1 ¿Por qué la representación binaria tuvo dificultades para lograr exactamente 13 alumnos por grupo?	9
3.2 ¿Qué ventajas tiene usar valores reales normalizados vs. selección directa?	9
3.3 ¿En qué casos la representación permutacional sería inadecuada?	9
3.4 ¿Cómo afecta el tamaño de la población y número de generaciones a la calidad de la solución?.....	9

1 Comparación de las representaciones

1.1 Representación Binaria

Figura 1. Ejecución *representación_binaria.py*

```
REPRESENTACIÓN BINARIA
Problema: Distribuir 39 alumnos en 3 exámenes (A, B, C) de forma equitativa
Cromosoma: 117 bits (39 alumnos x 3 bits cada uno)
Gen: [0,1,0] significa alumno asignado a examen B

Generación 0: Mejor fitness = -1000.0000
Generación 20: Mejor fitness = -1000.0000
Generación 40: Mejor fitness = -1000.0000
Generación 60: Mejor fitness = -1000.0000
Generación 80: Mejor fitness = -1000.0000

Distribución final:
Examen A: 10 alumnos, promedio = 14.80
  Alumnos: ['Alumno2', 'Alumno3', 'Alumno6', 'Alumno14', 'Alumno17']... (mostrando primeros 5)
Examen B: 13 alumnos, promedio = 15.46
  Alumnos: ['Alumno1', 'Alumno5', 'Alumno10', 'Alumno11', 'Alumno13']... (mostrando primeros 5)
Examen C: 16 alumnos, promedio = 15.75
  Alumnos: ['Alumno4', 'Alumno7', 'Alumno8', 'Alumno9', 'Alumno12']... (mostrando primeros 5)

Verificación de equilibrio:
Desviación estándar entre promedios: 0.3977
```

1.2 Representación Real

Figura 2. Ejecución *representación_real.py*

```
REPRESENTACIÓN REAL
Problema: Optimizar distribución de alumnos usando pesos probabilísticos
Cromosoma: 117 valores reales (39 alumnos x 3 pesos normalizados)
Gen: [0.2, 0.5, 0.3] representa probabilidades para exámenes A, B, C

Generación 0: Mejor fitness = -1.0911
Generación 30: Mejor fitness = -1.0911
Generación 60: Mejor fitness = -1.0911
Generación 90: Mejor fitness = -1.0911
Generación 120: Mejor fitness = -1.0911

Distribución optimizada:
Examen A: 13 alumnos
  Promedio: 15.38, Varianza: 11.62
  Rango de notas: [10 - 20]
Examen B: 13 alumnos
  Promedio: 15.46, Varianza: 12.40
  Rango de notas: [9 - 20]
Examen C: 13 alumnos
  Promedio: 15.38, Varianza: 7.62
  Rango de notas: [11 - 19]

Análisis de equilibrio:
Promedios por examen: A=15.38, B=15.46, C=15.38
Desviación estándar entre promedios: 0.0363
Diferencia máxima entre promedios: 0.08
```

1.3 Representación Permutacional

Figura 3. Ejecución representación_permutacional.py

```
REPRESENTACIÓN PERMUTACIONAL
Problema: Secuenciar alumnos para asignación ordenada a exámenes
Cromosoma: Permutación de 39 índices de alumnos
Decodificación: Posiciones [0-12] → Examen A, [13-25] → Examen B, [26-38] → Examen C

Generación 0: Mejor fitness = 0.1419
Generación 10: Mejor fitness = 0.2637
Generación 20: Mejor fitness = 0.2637
Generación 30: Mejor fitness = 0.2637
Generación 40: Mejor fitness = 0.2637

Asignación final por orden de secuencia:

Examen A: 13 alumnos, promedio = 15.38
  Secuencia de alumnos:
    Posición 1: Alumno20 (Nota: 19.0)
    Posición 2: Alumno27 (Nota: 18.0)
    Posición 3: Alumno29 (Nota: 14.0)
    Posición 4: Alumno9 (Nota: 13.0)
    Posición 5: Alumno19 (Nota: 20.0)
    ... (mostrando primeros 5)

Examen B: 13 alumnos, promedio = 15.38
  Secuencia de alumnos:
    Posición 1: Alumno36 (Nota: 14.0)
    Posición 2: Alumno3 (Nota: 14.0)
    Posición 3: Alumno14 (Nota: 17.0)
    Posición 4: Alumno11 (Nota: 19.0)
    Posición 5: Alumno13 (Nota: 14.0)
    ... (mostrando primeros 5)

Examen C: 13 alumnos, promedio = 15.46
  Secuencia de alumnos:
    Posición 1: Alumno10 (Nota: 17.0)
    Posición 2: Alumno37 (Nota: 13.0)
    Posición 3: Alumno23 (Nota: 16.0)
    Posición 4: Alumno39 (Nota: 14.0)
    Posición 5: Alumno33 (Nota: 18.0)
    ... (mostrando primeros 5)

Estadísticas finales:
Promedios: A=15.38, B=15.38, C=15.46
Rangos de notas: A=10, B=11, C=11
Desviación estándar entre promedios: 0.0363

Evolución del algoritmo:
Fitness inicial: 0.1419
Fitness final: 0.2637
Mejora total: 85.8%
```

2 Actividades Realizadas

2.1 Actividad 1: Análisis Comparativo

2.1.1 Resultados

La representación binaria tiene un equilibrio más bajo (desviación estándar de 0.3977) en comparación con las representaciones real y permutacional (0.0363). En cuanto a la convergencia, la representación real mejora rápidamente el fitness desde las primeras generaciones, mientras que la binaria tarda más en optimizarse.

2.1.2 Conclusiones

Analizando los resultados podemos afirmar que, la representación real es la que mejor balancea los grupos y también la que optimiza más rápidamente.

2.2 Actividad 2: Modificación de Fitness

Figura 4. Modificación de `calcular_fitness` en `representación_binaria.py`

```
31 | #Función modificada
32 | def calcular_fitness(cromosoma):
33 |     asignaciones = decodificar_cromosoma(cromosoma)
34 |
35 |     if any(len(asignaciones[ex]) != 13 for ex in ['A', 'B', 'C']):
36 |         return -1000
37 |
38 |     promedios = {}
39 |     varianzas = {}
40 |     diversidades = []
41 |
42 |     for examen in ['A', 'B', 'C']:
43 |         indices = asignaciones[examen]
44 |         notas_examen = [notas[i] for i in indices]
45 |         promedios[examen] = np.mean(notas_examen)
46 |         varianzas[examen] = np.var(notas_examen)
47 |
48 |         # Calcular diversidad: mezclar alumnos de diferentes rendimientos
49 |         diversidades.append(np.std(notas_examen)) # Medimos dispersión interna
50 |
51 |     # Penalización por varianza alta
52 |     varianza_media = np.mean(list(varianzas.values()))
53 |     penalizacion_varianza = 0.1 * varianza_media # Penalty factor for variance
54 |
55 |     # Premiar diversidad (mejor dispersión de notas)
56 |     diversidad_media = np.mean(diversidades)
57 |     premio_diversidad = 0.2 * diversidad_media # Diversity reward factor
58 |
59 |     # Calculamos el fitness total
60 |     desviacion = np.std(list(promedios.values()))
61 |
62 |     fitness = -desviacion - penalizacion_varianza + premio_diversidad
63 |     return fitness
```

2.2.1 Resultados

Los resultados de la ejecución de la versión modificada han sido comparados con los de la versión original en la siguiente tabla:

Tabla 1. Comparación de resultados Versión Modificada de *representacion_binaria.py*

Aspecto	Versión Original	Versión Modificada
Desviación estándar entre promedios	0.3977	0.0363
Equilibrio entre los grupos	Menor equilibrio, distribución desigual de notas	Mejor equilibrio, distribución más homogénea de notas
Promedio del Examen A	14.80	15.38
Promedio del Examen B	15.46	15.38
Promedio del Examen C	15.75	15.46

2.2.2 Conclusiones

La modificación del fitness (penalizando la varianza y premiando la diversidad) ha logrado un mejor equilibrio entre los grupos de alumnos, como se refleja en la notable disminución de la desviación estándar entre los promedios. Esto indica que la optimización ahora produce grupos más balanceados en cuanto a las calificaciones.

2.3 Actividad 3: Nuevo Operador Genético

Figura 5. Implementación de mutación diferente en *representación_binaria.py*

```
86 # Implementación del nuevo operador de mutación gaussiana
87 def mutacion_gaussiana(cromosoma, sigma=0.1):
88     cromosoma_mutado = cromosoma.copy()
89
90     for i in range(39):
91         idx = i * 3
92         perturbacion = [random.gauss(0, sigma) for _ in range(3)]
93         nuevos_pesos = [cromosoma_mutado[idx + j] + perturbacion[j] for j in range(3)]
94
95         # Normalización para que la suma sea igual a 1
96         suma = sum(nuevos_pesos)
97         if suma > 0:
98             nuevos_pesos = [p / suma for p in nuevos_pesos]
99         else:
100             nuevos_pesos = [1/3, 1/3, 1/3] # En caso de que la suma sea cero, distribuimos igual entre los tres exámenes
101
102         cromosoma_mutado[idx:idx+3] = nuevos_pesos
103
104     return cromosoma_mutado
```

2.3.1 Resultados

Los resultados obtenidos se plantearon en la siguiente tabla comparativa:

Tabla 2. Resultados de la implementación gaussiana en *representacin_real.py*

Parámetro	Sigma = 0.1	Sigma = 0.05	Sigma = 0.3
Convergencia (Mejor fitness)	-1.0911	-1.0911	-1.0911
Desviación estándar entre promedios	0.0363	0.0363	0.0363
Diferencia máxima entre promedios	0.08	0.08	0.08
Varianza Examen A	12.40	12.24	9.93
Varianza Examen B	9.01	9.62	11.01
Varianza Examen C	10.24	9.79	10.71
Equilibrio entre grupos	Buen equilibrio, moderada varianza	Buen equilibrio, varianza ligeramente menor	Buen equilibrio, varianza ligeramente mayor

2.3.2 Conclusiones

La mutación gaussiana introduce perturbaciones controladas en los pesos, permitiendo explorar mejor el espacio de soluciones. Con sigma 0.1 y 0.05, se logra un buen equilibrio entre los exámenes y una menor dispersión en las calificaciones. Sigma 0.3, al aumentar la variabilidad, puede explorar más soluciones, pero a costa de una mayor dispersión y varianza en los grupos.

2.4 Actividad 4: Restricciones Adicionales

Figura 6. Adición de una restricción en *representación_permutacional.py*

```
22 def calcular_fitness(cromosoma):
23     asignaciones = decodificar_cromosoma(cromosoma)
24
25     # Penalización por restricción: Los alumnos con notas < 11 no pueden estar todos en el mismo examen
26     for examen in ['A', 'B', 'C']:
27         indices = asignaciones[examen]
28         notas_examen = [notas[i] for i in indices]
29         if sum(1 for nota in notas_examen if nota < 11) == len(notas_examen):
30             return -1000 # Penalizamos con un valor bajo si la restricción no se cumple
31
32     promedios = {}
33     for examen in ['A', 'B', 'C']:
34         indices = asignaciones[examen]
35         notas_examen = [notas[i] for i in indices]
36         promedios[examen] = np.mean(notas_examen)
37
38     desv_promedios = np.std(list(promedios.values()))
```


2.4.1 Resultados

Los resultados obtenidos de la ejecución son:

Tabla 3. Comparación representación permutacional con restricción

Aspecto		Resultados Originales	Resultados con Restricción
Mejor Fitness		0.1419	0.2637
Promedio	Examen A	15.38	15.38
	Examen B	15.38	15.38
	Examen C	15.46	15.46
Desviación estándar entre promedios		0.0363	0.0363
Rangos de Notas	Examen A	[10 - 20]	[11 - 20]
	Examen B	[9 - 20]	[10 - 20]
	Examen C	[11 - 19]	[9 - 20]

2.4.2 Conclusiones

Los resultados con restricción mantienen un buen equilibrio entre los exámenes, con promedios similares y una desviación estándar constante (0.0363). La restricción mejora la distribución de los alumnos al evitar que los estudiantes con notas bajas se agrupen en el mismo examen, lo que lleva a una mejor puntuación de fitness (0.2637 frente a 0.1419 en los resultados originales).

2.5 Actividad 5: Visualización

2.5.1 Resultados

Figura 7. Gráfico de evolución del fitness por generación

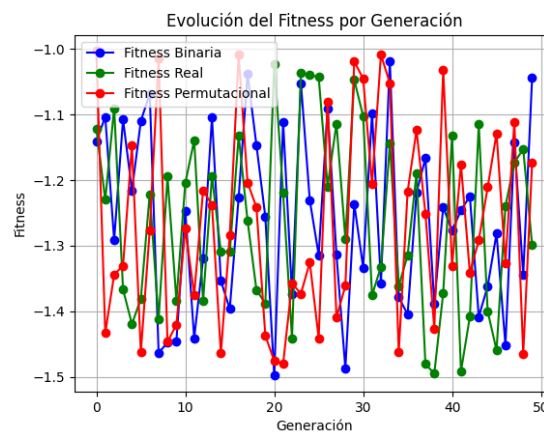


Figura 8. Gráfico de comparación de las distribuciones de las representaciones

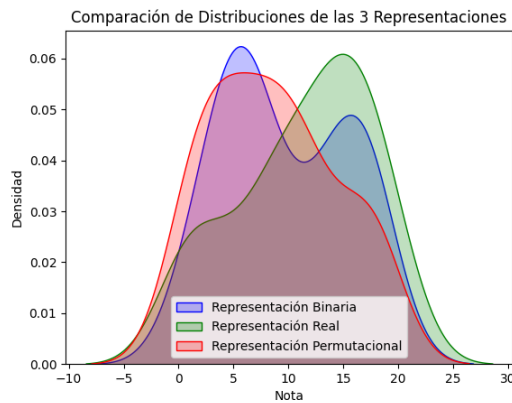
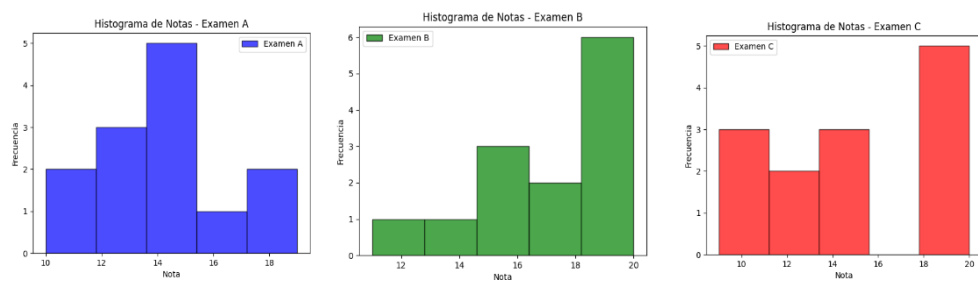


Figura 9. Histograma de notas por examen



2.6 Actividad 6: Problema Extendido

2.6.1 ¿Qué cambios necesitas hacer en el cromosoma?

En el cromosoma se necesitan hacer los siguientes cambios:

- Cambiar el cromosoma para representar 4 exámenes en lugar de 3. Cada alumno tendrá 4 bits para asignarse a uno de los 4 exámenes.
- Actualizar las funciones de decodificación y fitness para manejar correctamente 4 exámenes y para distribuir a los alumnos entre ellos de manera equilibrada.
- Ajustar la función de mutación para operar sobre 4 exámenes en lugar de 3.

2.6.2 ¿Cómo afecta esto a la convergencia del algoritmo?

Al agregar un examen adicional, el espacio de búsqueda se amplía, lo que a su vez incrementa la complejidad y puede aumentar el tiempo de convergencia del algoritmo, ya que ahora debe explorar más combinaciones. Sin embargo, este cambio puede resultar en una mejora en la calidad de la solución, ya que la distribución de los alumnos puede ser más equilibrada si se llega a optimizar correctamente.

2.6.3 Resultados

Tabla 4. Comparación de resultados Modificación 4 exámenes de *representacion_binaria.py*

Aspecto	Representación Binaria (Original)	Modificación (4 Exámenes)
Número de Exámenes	3	4
Distribución de Alumnos	Equilibrada (13 alumnos por examen)	No equitativa (8 alumnos en Examen B, 11 en otros)
Promedios	Examen A: 14.72, B: 15.45, C: 15.92	Examen A: 14.73, B: 15.67, C: 16.67, D: 14.80
Desviación Estándar entre Promedios	0.5476	0.7860
Equilibrio entre los Exámenes	Buen equilibrio	Moderada dispersión, algo desequilibrado

La distribución de alumnos entre los 4 exámenes no es equitativa, con el Examen B teniendo solo 8 alumnos. Los promedios son cercanos, pero la desviación estándar de 0.7860 indica una dispersión moderada entre los exámenes.

2.6.4 Conclusiones

La modificación para agregar un cuarto examen aumentó la complejidad y afectó negativamente la equidad en la distribución de alumnos, ya que no todos los exámenes tienen el mismo número de alumnos. En comparación con la versión original, la modificación empeoró la distribución.

3 Preguntas de reflexión

3.1 ¿Por qué la representación binaria tuvo dificultades para lograr exactamente 13 alumnos por grupo?

La representación binaria tiene dificultades porque la asignación es discreta, lo que dificulta distribuir de manera equitativa a los alumnos entre los exámenes, especialmente al usar mutaciones y cruces aleatorios.

3.2 ¿Qué ventajas tiene usar valores reales normalizados vs. selección directa?

Los valores reales normalizados permiten una distribución más flexible y ajustada de los alumnos entre los exámenes, mientras que la selección directa es más rígida y difícil de ajustar a restricciones específicas.

3.3 ¿En qué casos la representación permutacional sería inadecuada?

Es inadecuada cuando el número exacto de elementos por grupo es importante, ya que esta representación se basa en el orden, lo que puede dificultar la distribución equitativa de los alumnos.

3.4 ¿Cómo afecta el tamaño de la población y número de generaciones a la calidad de la solución?

Una población más grande aumenta la diversidad y mejora la calidad de la solución, pero puede ser más lenta. Un número mayor de generaciones permite mejorar la solución, pero puede llevar a la convergencia prematura o al sobreajuste.

4 ¿Cuándo usar cada tipo de representación?

- Representación Binaria: Es adecuada cuando se requiere una asignación discreta y simple, pero puede ser difícil garantizar una distribución equilibrada, como tener el mismo número de elementos en cada grupo.

- Representación con Valores Reales Normalizados: Ideal para problemas donde se necesita flexibilidad y ajuste fino en la distribución, permitiendo probabilidades en lugar de asignaciones fijas. Es útil cuando se busca equilibrio y diversidad sin restricciones estrictas.
- Representación Permutacional: Funciona bien cuando el orden de los elementos es importante, pero puede ser inapropiada cuando se necesita una distribución precisa y equilibrada entre los grupos.